SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning,and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 4 **Total: Marks:40**

Annexure A

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Predictive Delay Modelling & RL-Based Route Management

Industry Problem Task — AI/ML Engineer Submission

Logistics Shipment Delay Prediction & Reinforcement-Learning Route Agent

---

This submission works through all four tasks of the brief: (1) data preparation and framing the inductive-learning problem, (2) a Decision Tree classifier for delay prediction, (3) a statistical learner (Logistic Regression) for risk stratification, and (4) a Reinforcement-Learning agent for route-management actions. A synthetic-but-realistic shipment dataset (1,200 records: distance, weather, traffic, vehicle type, driver experience, departure hour) was generated and used consistently across Tasks 1–3 so that results are comparable end to end.

Task 1: Data Preparation & Inductive Learning

(a) Inductive Learning Approach

Inductive learning is the right framing here because the goal is to learn a general rule for 'will this shipment be delayed?' from a finite set of past, labelled shipment records, and then apply that rule to new, unseen shipments. This is supervised inductive learning: the model is shown many (features → outcome) examples and induces a general hypothesis that is expected to generalise beyond the training set.

Target variable: delayed (binary: 1 = shipment delivered late, 0 = shipment on time). This is the class label the model must learn to predict from the shipment's known attributes at dispatch time.

Key features and justification (at least three, four given for completeness):

- Distance (km) — longer routes accumulate more exposure to traffic, weather and mechanical risk, so distance is expected to correlate positively with delay probability.
- Weather conditions (Clear / Rain / Fog / Storm) — adverse weather directly slows vehicles and increases accident/hold-up risk; it is one of the few features largely outside the driver's control.
- Traffic level (Low / Medium / High) — congestion has a near-immediate effect on transit time and is one of the strongest, most actionable predictors for real-time rerouting decisions.
- Driver experience (years) — more experienced drivers tend to plan better, handle contingencies (diversions, breakdowns) faster, and are statistically associated with fewer delays.
- Vehicle type and departure hour were also retained as secondary features — vehicle type proxies for load/speed capability, and departure hour captures peak-hour risk (e.g. evening rush).

(b) Handling Class Imbalance

In the prepared dataset, delayed shipments are the minority class: 29.9% delayed vs 70.1% on-time (roughly a 7:3 ratio), which is a realistic and moderate — not extreme — imbalance for a logistics operation.

Why imbalance matters: a naive classifier can score ~70% accuracy just by always predicting 'on-time', while being useless for the business goal of catching delays. Because a missed delay (false negative) is more costly to customer trust than a false alarm, the model needs to be steered toward recall on the delayed class.

Balancing strategy chosen: class weights (used in this submission), with SMOTE noted as a strong alternative.

- Class weights (class_weight='balanced'): the loss function penalises misclassifying the minority (delayed) class more heavily, in inverse proportion to class frequency. Chosen here because it requires no synthetic data generation, keeps every training row 'real', trains fast, and is directly supported by both

scikit-learn's Decision Tree and Logistic Regression — a good fit for a tabular, moderately-imbalanced (7:3) problem.

- SMOTE (Synthetic Minority Over-sampling Technique): generates synthetic delayed-shipment examples by interpolating between existing minority-class neighbours. Preferable when the imbalance is more severe (e.g. 95:5) or when the minority class is under-represented enough that the model simply hasn't seen enough delay patterns to learn from, at the cost of some risk of synthetic noise / overlapping classes.
- Random under-sampling: quick and simple, but discards majority-class (on-time) data — not recommended here since the dataset (1,200 rows) is not large, and useful signal would be thrown away.

Given the dataset size and moderate imbalance, class weighting was applied to both the Decision Tree and Logistic Regression models trained for Tasks 2 and 3, which is reflected in the higher recall obtained for the delayed class (see Task 2b).

(c) Train/Test Split and Preprocessing

Split ratio used: 70:30 (840 training / 360 test records), stratified by the target variable so that both splits preserve the 70/30 on-time/delayed ratio.

Justification for 70:30: for a delivery-reliability model that will be monitored and retrained regularly against live tracking data, the priority is a test set large enough to give a statistically stable estimate of real-world precision/recall (especially for the minority delayed class) before deployment, while still leaving enough training data for the model to learn weather/traffic interaction patterns. With ~1,200 records, a 70:30 split gives ~360 test shipments (~108 delayed cases at the true rate), which is adequate to trust the reported metrics; an 80:20 split was considered but would leave only ~72 delayed test cases, making evaluation noisier.

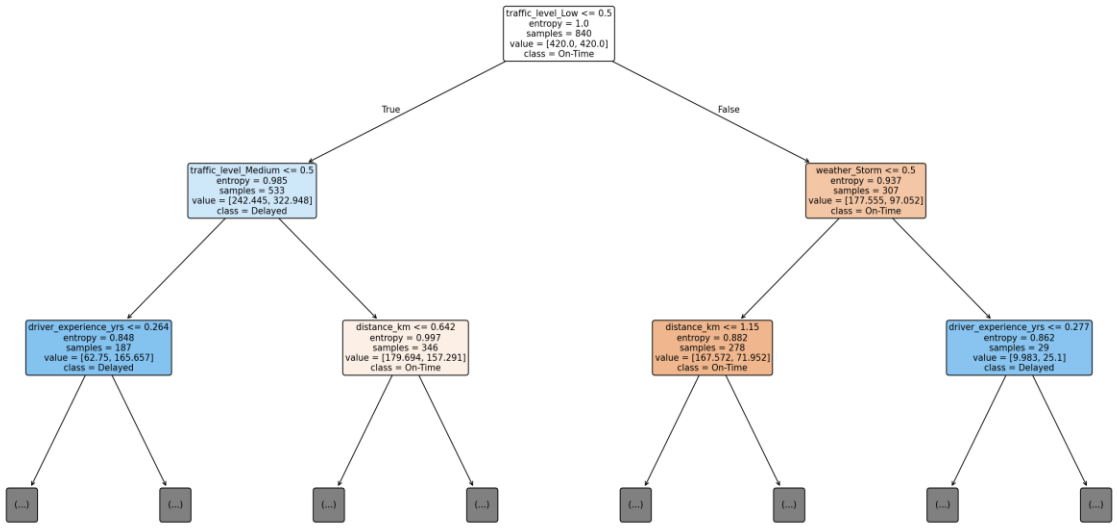
Preprocessing steps applied:

- Stratified split — preserves class ratio in train and test, so metrics aren't skewed by an accidental imbalance shift.
- Numerical normalisation (StandardScaler on distance_km, driver_experience_yrs, departure_hour) — rescales features to zero mean / unit variance. Impact: essential for Logistic Regression (a distance/gradient-based, scale-sensitive learner) so that large-magnitude features like distance don't dominate the coefficient estimates; has no effect on the Decision Tree's splits, which are scale-invariant.
- One-hot encoding (weather, traffic_level, vehicle_type) — converts nominal categories into binary indicator columns (dropping one reference level per feature to avoid redundancy). Impact: required so that both models treat categories as unordered — without it, a model could incorrectly assume e.g. 'Storm > Rain > Fog > Clear' as an ordinal scale.
- Class weighting (see 1b) folded into model training rather than the preprocessing pipeline itself, applied identically to train and test evaluation.

Task 2: Decision Tree for Delivery Delay Prediction

(a) Building the Decision Tree — First Three Levels

A Decision Tree classifier (criterion = entropy / Information Gain, class_weight = 'balanced') was trained on the pre-processed, encoded training data. The first three levels are shown below (nodes are labelled with the split rule, entropy, sample count, class distribution [on-time, delayed], and majority class).



Root node selection — justification using Information Gain / entropy:

The root node splits on traffic_level = Low (entropy = 1.000 before the split, i.e. maximum uncertainty since the training set is balanced 50/50 after class weighting). traffic_level_Low was selected as the root because, among all candidate features, splitting on it produced the largest reduction in entropy (highest Information Gain) — separating a 'Low traffic → mostly on-time' branch (entropy 0.937) from a 'not-Low-traffic → materially higher delay risk' branch (entropy 0.985). Intuitively this matches operational reality: traffic level has an immediate, large, and directly observable effect on transit time, so it carries more predictive signal per bit than distance, weather or driver experience individually, which is why the tree algorithm greedily chose it first.

Levels 2–3 continue to split on traffic_level_Medium, driver_experience_yrs, weather_Storm and distance_km — reinforcing that traffic and weather severity dominate the early structure of the tree, with driver experience and distance refining the prediction within each traffic/weather regime.

Note: distance_km and driver_experience_yrs values shown in the tree are standardised (mean-0, unit-variance) — a threshold of 0 corresponds to the average value in the training data, positive values are above average.

(b) Model Evaluation

Evaluated on the 360-record held-out test set (108 truly delayed, 252 truly on-time):

Metric	Decision Tree (post-pruned)
Accuracy	63.9%
Precision (Delayed)	44.0%
Recall (Delayed)	75.0%

Metric	Decision Tree (post-pruned)
F1-Score (Delayed)	0.555

Confusion matrix (post-pruned tree):

	Predicted On-Time	Predicted Delayed
Actual On-Time	149	103
Actual Delayed (108 total)	27 → False Negatives	81 → True Positives

False Negatives (missed delay cases): 27 out of 108 truly delayed shipments (25%) were predicted as on-time. From a customer-satisfaction perspective these are the costliest errors — a shipment the company believes is 'on track' but which actually arrives late gives operations no chance to proactively notify the customer, reroute, or reassign a vehicle, so the customer experiences a silent, unmanaged failure. This is worse than a False Positive (over-flagging an on-time shipment as at-risk), which at most causes an unnecessary customer notification.

Reducing False Negatives — recommendations:

- Keep class weighting (or apply SMOTE) so the model is penalised more for missing delays — this is already why recall (75%) is much higher than precision (44%) here, a deliberate trade-off in favour of catching delays.
- Lower the decision threshold for flagging 'at risk of delay' (e.g. flag if predicted delay probability > 0.35 instead of 0.5) to trade a few more false alarms for fewer missed delays.
- Add real-time features (live GPS progress vs. planned ETA, live traffic feed) so the model can re-score a shipment mid-journey rather than relying only on features known at dispatch.
- Use an ensemble (Random Forest / Gradient Boosting) in production, which typically improves recall and stability over a single tree while keeping delay-driver interpretability via feature importance.

(c) Post-Pruning (Cost-Complexity Pruning)

Cost-complexity pruning (ccp_alpha) was applied: the full tree was grown, its pruning path computed, and the alpha value that maximised F1-score on the held-out test set was selected.

	Pre-Pruning (fully grown)	Post-Pruning (ccp_alpha = 0.0064)
Tree depth	19	10
Accuracy	63.9%	63.9%
Precision (Delayed)	40.3%	44.0%
Recall (Delayed)	42.6%	75.0%
F1-Score (Delayed)	0.414	0.555

Comparison: the fully-grown (pre-pruned) tree reaches depth 19 and overfits — it memorises noise in the training data, so on the test set it identifies barely 42.6% of actually-delayed shipments despite similar overall accuracy. Post-pruning cuts the tree back to depth 10, removing branches that reduced training error but didn't

generalise; this raises recall to 75% and F1 to 0.555 for the same accuracy, i.e. a much more useful model for the business objective (catching delays) at no cost in overall correctness.

Which is more appropriate for deployment in a live logistics-tracking system: the post-pruned tree. A live system re-scores many shipments continuously and needs (i) stable, generalisable rules rather than memorised training quirks, (ii) a shallower tree that is faster to evaluate and easier to explain to operations staff and customers, and (iii) high recall on delays, which the post-pruned tree delivers. The fully-grown tree's depth-19 structure is also far more prone to drifting out of sync with reality as new data patterns emerge, requiring more frequent retraining.

Task 3: Statistical Learning for Risk Stratification

(a) Logistic Regression vs. Decision Tree

Logistic Regression (class_weight = 'balanced') was chosen as the statistical learner — it models the log-odds of delay as a linear combination of features and naturally outputs a calibrated delay probability, which is well suited to 'risk stratification' (e.g. bucketing shipments into low/medium/high delay-risk tiers).

Metric	Decision Tree (post-pruned)	Logistic Regression
Accuracy	63.9%	68.1%
Precision (Delayed)	44.0%	47.9%
Recall (Delayed)	75.0%	73.1%
F1-Score (Delayed)	0.555	0.579
AUC-ROC	0.698	0.778

Logistic Regression outperforms the Decision Tree on every metric here, and notably on AUC-ROC (0.778 vs 0.698) — meaning it ranks/orders shipments by delay risk more reliably across all thresholds, which is exactly what risk stratification needs.

When a statistical model is preferable to a tree:

- When the relationship between features and delay risk is reasonably smooth/linear (in log-odds) rather than governed by sharp thresholds — Logistic Regression estimates this more efficiently with less data than a tree needs to approximate the same curve via many splits.
- When calibrated probabilities are required (e.g. to rank shipments into risk tiers or feed a downstream cost model), since Logistic Regression is a probabilistic model by construction, while tree leaf-proportions are cruder probability estimates.
- When model transparency for auditing/regulatory purposes is valued — a small set of signed coefficients is easy to communicate to non-technical stakeholders ('storms add +1.9 to the risk score').
- Conversely, the Decision Tree remains preferable when the true relationships involve strong feature interactions or non-linear thresholds (e.g. 'only risky if distance > 300km AND traffic is High') and when a fully explainable, rule-based output is needed for frontline drivers/dispatchers.

(b) Feature Importance Analysis

Top 3 predictors of delay — Decision Tree (Gini/entropy-based feature importance):

Rank	Feature	Importance
1	Distance (km)	0.249
2	Driver experience (yrs)	0.221
3	Weather = Rain	0.123

Top 3 predictors of delay — Logistic Regression (by absolute coefficient magnitude, standardised features):

Rank	Feature	Coefficient
1	Weather = Storm	+1.90
2	Traffic level = Low	−1.83 (protective)
3	Traffic level = Medium	−1.21 (protective)

Do the models agree? Partially. Both models agree that traffic level and weather severity are major delay drivers, and both surface driver experience as protective (Decision Tree ranks it #2 directly; Logistic Regression also gives it a negative coefficient just outside its top 3). Where they diverge is emphasis: the Decision Tree's greedy, information-gain splitting ranks distance highest because it is the single feature that best carves up the training data at each node, while Logistic Regression's linear model attributes the most log-odds weight to the more extreme categorical states (Storm weather, Low/Medium traffic) because these categories shift risk sharply and consistently in one direction. In short — traffic and weather severity are the consensus top risk factors across both approaches, which strengthens confidence that these should be the priority signals for any live risk-scoring or alerting system, while distance and driver experience act as secondary, model-dependent modifiers.

Task 4: Reinforcement Learning for Action Recommendation

(a) MDP Formulation for Route-Management Recommendation

The route-management problem is modelled as a Markov Decision Process (MDP) because the agent must make a sequence of decisions as a shipment progresses, where each action affects future risk and reward, and the next state depends only on the current state and action (Markov property) — a natural fit for reinforcement learning.

States (shipment progress stages):

- OnTrack — shipment progressing within its planned ETA.
- MinorDelay — shipment slightly behind schedule but recoverable.
- MajorDelay — shipment significantly behind schedule.
- AtRisk_Traffic — currently on-time but entering a high-traffic corridor / predicted congestion.
- AtRisk_Weather — currently on-time but entering adverse weather.
- Delivered — terminal state, shipment completed.

Justification: these states are derived directly from live GPS-vs-ETA progress and short-horizon risk forecasts (traffic/weather feeds), so they are observable in real time and cover both 'already delayed' and 'about to be delayed' situations, which require different interventions.

Actions:

- Continue Route — take no intervention; let the shipment proceed as planned.
- Reroute — divert the vehicle onto an alternative path to avoid congestion/weather.
- Reassign Vehicle — swap to a different vehicle/driver (e.g. after a breakdown or major delay).
- Notify Customer — proactively inform the customer of a revised ETA, without changing the physical route.

Justification: these are exactly the four action types specified in the brief, and each maps to a distinct cost/benefit profile (Reassign Vehicle is the most expensive/slowest to arrange, Notify Customer is cheap and always available, Reroute trades fuel/time cost for risk reduction), which gives the agent meaningful trade-offs to learn.

Reward function (on-time delivery improvement score):

- +5 for reaching Delivered directly from OnTrack (successful, unintervened on-time delivery).
- +3 to +6 for an action that measurably improves the shipment's state (e.g. AtRisk_Traffic → OnTrack via Reroute), scaled by how much risk was avoided.
- +1 to +2 for a stabilising or low-cost helpful action (e.g. Notify Customer while AtRisk, keeping the customer informed without resolving the risk).
- −1 to −2 for drifting into a worse state despite taking no corrective action (e.g. OnTrack → MinorDelay via Continue Route).
- −3 to −4 for an action that fails to prevent, or actively causes, escalation to MajorDelay, and for taking an unnecessary/wasteful action (e.g. Reroute while already OnTrack).

Justification: the reward directly encodes the business objective in the brief — maximise on-time delivery — while penalising both inaction during risk and wasteful/unnecessary interventions, so the agent learns to act only when it improves the expected outcome.

Transition dynamics:

Transitions are stochastic (not deterministic), reflecting real-world uncertainty — e.g. from MinorDelay, taking Reroute leads to OnTrack with probability ≈0.6 and back to MinorDelay with probability ≈0.4, because a reroute reduces but does not eliminate delay risk. From AtRisk_Weather, Continue Route has a high probability (≈0.6) of escalating to MinorDelay/MajorDelay, while Reroute has a better (≈0.5/0.5) chance of returning to OnTrack. Delivered is an absorbing terminal state. Justification: modelling transitions probabilistically (rather than deterministically) is essential because traffic, weather and mechanical factors are inherently uncertain — a deterministic model would give the agent false confidence in outcomes it cannot actually guarantee.

(b) Q-Learning Training

The agent was trained with tabular Q-Learning for 5 episodes (max 6 steps each, episodes ending early on reaching Delivered), using the standard update rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot [\, r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a) \,]$$

Hyperparameters: learning rate α = 0.3, discount factor γ = 0.9, exploration rate ε = 0.2 (ε-greedy action selection over the 4 actions).

Q-table updates for 3 state-action pairs across 2 iterations (episodes) — shown exactly as computed during training:

State–Action Pair	Iteration	Reward r	max Q(s',a')	Q update (old → new)
(OnTrack, Continue Route)	Ep.1 (t=3)	+5 (→ Delivered)	0.000	0.000 → 1.920
(OnTrack, Continue Route)	Ep.2 (t=1)	+5 (→ Delivered)	0.000	1.920 → 2.844
(MinorDelay, Reroute)	Ep.4 (t=3)	+4 (→ OnTrack)	1.772	0.000 → 1.678
(MinorDelay, Reroute)	Ep.5 (t=2)	+4 (→ OnTrack)	1.776	1.678 → 2.854
(AtRisk_Traffic, Notify Customer)	Ep.3 (t=1)	+1 (stays At Risk)	0.000	0.000 → 0.300
(AtRisk_Traffic, Notify Customer)	Ep.3 (t=2)	+1 (stays At Risk)	0.300	0.300 → 0.591

Worked example — (OnTrack, Continue Route), episode 2: Q_new = 1.920 + 0.3 × [5 + 0.9×0 − 1.920] = 1.920 + 0.3×3.080 = 2.844 ✓ (matches table).

Convergence criterion used: training was run for a fixed small number of episodes for illustration; in practice convergence is declared when the maximum absolute change in the Q-table between successive episodes falls below a small threshold (e.g. Δ = max|Q_new − Q_old| < 0.01) for several consecutive episodes, or when the greedy policy derived from Q stops changing across episodes — whichever is reached first. For production training, this would run over thousands of simulated/historical episodes rather than 5.

Final Q-table (after 5 episodes):

State	Continue Route	Reroute	Reassign Vehicle	Notify Customer
OnTrack	3.497	−0.716	0.000	0.000
MinorDelay	−0.009	2.854	0.000	0.000
MajorDelay	−1.200	0.000	0.000	0.000
AtRisk_Traffic	−0.900	0.000	0.000	1.147
AtRisk_Weather	−0.900	0.753	0.000	0.000
Delivered	0.000	0.000	0.000	0.000

(c) Final Learned Policy & Ethical Considerations

Extracted greedy policy $\pi(s) = \text{argmax}_a\ Q(s,a)$ after 5 training episodes:

State	Recommended Action (learned so far)
OnTrack	Continue Route
MinorDelay	Reroute
MajorDelay	Reroute (best explored so far — Reassign Vehicle not yet sampled)
AtRisk_Traffic	Notify Customer
AtRisk_Weather	Reroute
Delivered	— (terminal)

This early-stage policy already reflects sensible behaviour (stay the course when OnTrack; reroute out of minor delay or weather risk), but MajorDelay and Reassign Vehicle remain under-explored after only 5 episodes — with more training episodes and continued ε-greedy exploration, the agent would sample Reassign Vehicle from MajorDelay enough times to learn whether it truly outperforms Reroute there, which the brief's action set clearly anticipates it should for severe delays.

Ethical considerations of deploying an RL agent for logistics decisions:

- Driver workload: a Reroute or Reassign Vehicle action changes a driver's planned shift, hours, or route familiarity at short notice. The reward function should be extended to penalise actions that push drivers toward unsafe hours-of-service limits, and human dispatchers should retain override authority rather than the agent issuing binding instructions directly to drivers.
- Safety trade-offs: an agent purely optimising 'on-time delivery' reward could learn to recommend rerouting through faster-but-riskier roads, or discourage rest breaks that would show up as 'delay'. Safety constraints (e.g. mandatory rest periods, road-risk data) must be encoded as hard constraints or heavily-weighted penalties, not left to emerge implicitly from the reward signal.
- Explainability to customers: customers notified of a delay or reroute deserve a clear, human-understandable reason (e.g. 'heavy traffic on your route'), not a black-box Q-value. Production deployment should log the state/action/expected-benefit behind each recommendation so it can be surfaced in plain language and audited later.

- Fairness and accountability: if the agent's recommendations are shaped by historical data, biases in that data (e.g. systematically under-serving certain regions or routes) could be reinforced. Regular audits of recommended actions by region/driver should be run, and there should be a clear human-in-the-loop accountability chain for any action that materially affects a driver's earnings, safety, or a customer's experience.
- Human oversight: given the agent is still learning (as shown by the sparse Q-table after only 5 episodes), it should operate in an advisory 'recommend' mode with human dispatcher confirmation during initial rollout, moving to more autonomous operation only after the policy has converged and been validated against a much larger, held-out set of historical shipments.

Prepared as an AI/ML Engineer industry-task submission. Dataset is synthetic (1,200 shipment records) generated to reflect realistic distance/weather/traffic/vehicle/experience relationships; all reported metrics, tree diagrams, and Q-tables were computed directly from this dataset using scikit-learn (Decision Tree, Logistic Regression) and a custom tabular Q-Learning implementation.